

Questões Teóricas

1. O que significa declarar uma variável com var? Isso torna seu tipo dinâmico?

R: Significa que seu tipo não foi definido durante a codificação, e vai ser definido durante o tempo de compilação. Não, pois ele não pode ser alterado uma vez que é definido.

2. O que é conversão de tipos? Diferencie Parse de TryParse e explique a importância de validar uma entrada.

R: É a conversão de um valor que é de um tipo para outro tipo, um exemplo é a conversão de *string* para *int*. O *Parse* só tenta converter e se não der certo segue o código gera uma exceção, já o *TryParse* retorna um booleano (se foi bem-sucedido retorna *true*, senão retorna *false*), o que faz ser possível verificar e não deixar o código prosseguir enquanto não realizar a conversão.

3. O que são parâmetro opcional e argumento nomeado?

R: O parâmetro opcional, é possível escolher se vai ser passado ou não na chamada da função, e também ele já tem um valor pré-definido dentro na assinatura. Já o argumento nomeado é a especificação do parâmetro que vai ser passado, não sendo necessário respeitar a ordem dos itens.

4. Defina sobrecarga de métodos. Apresente duas assinaturas válidas de métodos sobrecarregados e explique o que as diferencia.

R: Sobrecarga de métodos é quando existem métodos com o mesmo nome mas podem variar nos tipos, quantidades ou ordem dos parâmetros. O que diferencia uma da outra neste caso abaixo, é a quantidade de parâmetros e o tipo, enquanto uma recebe dois valores do tipo *double*, a outra recebe apenas um *array* do tipo *double*.

```
public double soma (double a, double b);
```

```
public double soma (double[] valores);
```

5. O que é uma List<T>? Em que ela difere de um array quanto à quantidade de elementos?

R: Uma lista é uma coleção em que todos os dados têm o mesmo tipo, e ela é quase a mesma coisa de um array tirando o fato de não ter um tamanho exato. Ela pode conter zero elementos, assim como podem ser adicionados n elementos, e estes podem ser retirados a qualquer momento.

6. O que é um Dictionary<TKey, TValue>? Defina chave e valor, explique a unicidade das chaves e dê um exemplo de aplicação.

R: É uma coleção em que ao invés de termos posições (um *index*) para cada valor, uma chave de qualquer tipo (definido na instanciação) é utilizada para acessar o valor desejado (também tem o tipo definido na instanciação). A chave é o valor que usa-se como se fosse a indexação para acessar o valor. Cada chave não pode repetir, enquanto os valores podem. Um exemplo de aplicação é a matrícula de alunos, em que não podem existir dois alunos com a mesma matrícula.

7. Defina classe e objeto e explique a relação entre eles.

R: Classe é o molde e a abstração de uma coisa que existe na vida real, que possui suas características (atributos) e métodos. Já o objeto é a instância de uma variável em que seu tipo é uma classe, e que respeita suas regras e meios de interagir.

8. O que é um construtor? Explique sua função na criação de um objeto e diferencie um membro de instância de um membro static.

R: O construtor é uma função de uma classe que tem o mesmo nome dela e que pode ou não ser chamada durante a instanciação do objeto. Sua função na criação de um objeto é a possibilidade de definir ou não valores para os seus atributos. Um membro de instância pertence a um objeto da classe e pode acessar seu estado, já um membro *static* pertence à própria classe e pode ser chamado sem a necessidade de instanciar um objeto.

9. Explique os modificadores public, private e protected, além dos acessores get e set.

R: Os modificadores servem para controlar o encapsulamento dos atributos e métodos de uma classe. O modificador *public* permite o acesso por qualquer código que conheça o objeto; o *private* apenas dentro da própria classe; o *protected* na classe e em suas

classes derivadas. O *get* permite a leitura do valor que está na variável e o *set* permite a alteração desse valor.

10. Defina polimorfismo e explique os papéis de *virtual* e *override* na sobrescrita de métodos.

R: Polimorfismo é quando métodos realizam comportamentos adequados a cada objeto, um exemplo é dois animais que emitem som, mas cada um com seu jeito específico. *Virtual* serve para colocar na assinatura do método de uma classe para dizer que ele possui uma implementação padrão, mas pode ser sobrescrita pelas classes filhas. Já o *override* é usado na assinatura do método na classe filha, para indicar que ele está sobrescrevendo o método da classe pai.

11. O que é uma classe abstrata? O que é um método abstrato?

R: Uma classe abstrata não é instanciada, mas pode manter estado, possuir construtor e combinar membros concretos, virtuais e abstratos. Já um método abstrato é apenas uma garantia de que qualquer classe filha concreta terá que implementar esse método.

12. O que é uma interface? Diferencie seu papel do de uma classe abstrata e explique a ideia de contrato.

R: Uma interface é um contrato, ou seja, ela define o que uma classe deve saber fazer, mas não determina como. A sua diferença para a classe abstrata é que a classe abstrata pode manter estado e possuir construtores, enquanto a interface não possui estado. Além disso, uma classe pode implementar múltiplas interfaces, mas só pode herdar de uma única classe abstrata.

Questões Práticas

1. Calculadora de média:

```
using System.Globalization;

class Program
{
    static string separadorDoSistema =
CultureInfo.CurrentCulture.NumberFormat.NumberDecimalSeparator;
    static string LeEntrada() => Console.ReadLine() ?? "";

    static string LeEntradaNaoVazia(string mensagem)
    {
        while (true)
        {
            Console.Write(mensagem);
            string leitura = LeEntrada().Trim();
            if (!string.IsNullOrEmpty(leitura))
            {
                return leitura;
            }
            Console.WriteLine("\nValor inválido! O campo não pode ser
vazio.");
        }
    }

    static double LeNota(string mensagem)
    {
        while (true)
        {
            Console.Write(mensagem);
            string entrada = LeEntrada().Trim();
            entrada = separadorDoSistema == "," ? entrada.Replace('.',
',') : entrada.Replace(',', '.');

            if (!double.TryParse(entrada, out double resultado))
            {
                Console.WriteLine("\nERRO! Digite um valor válido!");
            }
            else if (resultado < 0 || resultado > 10)
            {
                Console.WriteLine("\nERRO! Digite um valor entre 0 a
```

```

10!");
        }
        else
        {
            return resultado;
        }
    }
}

static double CalcularMedia(double nota1, double nota2) => (nota1
+ nota2) / 2;

static void Main()
{
    Console.Clear();
    Console.WriteLine("=== CALCULADOR DE MÉDIA ===\n");
    Console.WriteLine("Este programa aceita vírgula ou ponto como
separador decimal.\n");

    while (true)
    {
        string nome = LeEntradaNaoVazia("Digite o nome do
estudante: ");
        double nota1 = LeNota("Digite a primeira nota: ");
        double nota2 = LeNota("Digite a segunda nota: ");

        double media = CalcularMedia(nota1, nota2);

        Console.WriteLine($"Nome....: {nome}");
        Console.WriteLine($"Média...: {media:F2}");
        Console.WriteLine($"Situação: {(media >= 6 ? "Aprovado" :
media >= 4 ? "Recuperação" : "Reprovado")}!\n");

        string opcao = LeEntradaNaoVazia("Deseja calcular outra
média? [s/n]: ").ToLower();
        if (opcao == "n")
        {
            Console.WriteLine("\n--> PROGRAMA ENCERRADO <--\n");
            break;
        }
    }
}

```

```
}
```

2. Análise de cinco números:

```
class Program
{
    static string LeEntrada() => Console.ReadLine() ?? "";

    static int LeInteiro(string mensagem)
    {
        while (true)
        {
            Console.Write(mensagem);
            string entrada = LeEntrada().Trim();

            if (int.TryParse(entrada, out int resultado))
            {
                return resultado;
            }
            Console.WriteLine("\nERRO! Digite um valor válido!");
        }
    }

    static int Somar(int[] numeros)
    {
        int soma = 0;
        for (int i = 0; i < numeros.Length; i++)
        {
            soma += numeros[i];
        }
        return soma;
    }

    static int EncontrarMaior(int[] numeros)
    {
        int maior = numeros[0];
        for (int i = 1; i < numeros.Length; i++)
        {
            if (numeros[i] > maior)
            {
                maior = numeros[i];
            }
        }
    }
}
```

```

    }
    return maior;
}

static double CalcularMedia(int[] numeros)
{
    return (double)Somar(numeros) / numeros.Length;
}

static int ContarNumerosPares(int[] numeros)
{
    int numerosPares = 0;
    for (int i = 0; i < numeros.Length; i++)
    {
        if (numeros[i] % 2 == 0)
            numerosPares++;
    }
    return numerosPares;
}

static string ExibirNaOrdemDeLeitura(int[] numeros)
{
    string resposta = $"{numeros[0]}";
    for (int i = 1; i < numeros.Length; i++)
    {
        resposta += $" -> {numeros[i]}";
    }
    return resposta;
}

static void Main()
{
    Console.Clear();
    Console.WriteLine("=== ANÁLISE DE CINCO NÚMEROS ===\n");

    int[] numeros = new int[5];
    for (int i = 0; i < 5; i++)
    {
        numeros[i] = LeInteiro($"Digite o {i + 1}º número: ");
    }

    Console.WriteLine($"
Números na ordem de leitura:

```

```

{ExibirNaOrdemDeLeitura(numeros)}");
    Console.WriteLine($"Soma dos números: {Somar(numeros)}");
    Console.WriteLine($"Média dos números:
{CalcularMedia(numeros):F2}");
    Console.WriteLine($"Maior dos números:
{EncontrarMaior(numeros)}");
    Console.WriteLine($"Quantidade de números pares:
{ContarNumerosPares(numeros)}");
}
}

```

3. Lista de compras:

```

class Program
{
    static string LeEntrada() => Console.ReadLine() ?? "";

    static string LeStringObrigatoria(string mensagem)
    {
        while (true)
        {
            Console.Write(mensagem);
            string leitura = LeEntrada().Trim();
            if (!string.IsNullOrEmpty(leitura))
            {
                return leitura;
            }
            Console.WriteLine("\nValor inválido! O campo não pode ser
vazio.");
        }
    }

    static int LerInteiro(string mensagem)
    {
        while (true)
        {
            Console.Write(mensagem);
            string entrada = LeEntrada().Trim();

            if (int.TryParse(entrada, out int resultado))
            {
                return resultado;
            }
        }
    }
}

```

```

        }
        Console.WriteLine("\nERRO! Digite um valor válido!");
    }
}

static int Menu()
{
    Console.WriteLine("\n=====> MENU <=====");
    Console.WriteLine("0 - Sair\t\t2 - Listar itens");
    Console.WriteLine("1 - Adicionar item\t3 - Remover item");
    Console.WriteLine("=====> MENU <=====");
    return LerInteiro("Digite sua opção: ");
}

static void AdicionarNome(List<string> lista)
{
    string novoNome = LeStringObrigatoria("Digite o item a ser
adicionado: ");
    lista.Add(novoNome);
    Console.WriteLine("\nITEM ADICIONADO COM SUCESSO!");
}

static void ListarNomes(List<string> lista)
{
    if (lista.Count == 0)
    {
        Console.WriteLine("ERRO! Lista está vazia.");
        return;
    }

    Console.WriteLine("\n-----> LISTA DE ITENS <-----");
    for (int i = 0; i < lista.Count; i++)
    {
        Console.WriteLine($"{i + 1}° -> {lista[i]}");
    }
    Console.WriteLine($"# QUANTIDADE TOTAL: {lista.Count}");
    Console.WriteLine("-----");
}

static void RemoverNome(List<string> lista)
{
    if (lista.Count == 0)
    {

```

```

        Console.WriteLine("ERRO! Lista está vazia.");
        return;
    }

    int posicaoParaRemover = LerInteiro("Digite a posição do item
a ser removido: ");
    if (posicaoParaRemover < 1 || posicaoParaRemover >
lista.Count)
    {
        Console.WriteLine("ERRO! Posição não existe.");
        return;
    }

    lista.RemoveAt(posicaoParaRemover - 1);
    Console.WriteLine("\nITEM REMOVIDO COM SUCESSO!");
}

static void Main()
{
    List<string> lista = new List<string>();
    while (true)
    {
        int opcao = Menu();
        switch (opcao)
        {
            case 0:
                Console.WriteLine("\n--> PROGRAMA ENCERRADO
<--\n");
                return;
            case 1:
                AdicionarNome(lista);
                break;
            case 2:
                ListarNomes(lista);
                break;
            case 3:
                RemoverNome(lista);
                break;
            default:
                Console.WriteLine("\nOPÇÃO INVÁLIDA\n");
                break;
        }
    }
}

```

```
}  
  
}  
  
}
```

4. Consulta de preços:

```
using System.Globalization;  
  
class Program  
{  
    static string separadorDoSistema =  
CultureInfo.CurrentCulture.NumberFormat.NumberDecimalSeparator;  
  
    static string LeEntrada() => Console.ReadLine() ?? "";  
  
    static string LeStringObrigatoria(string mensagem)  
    {  
        while (true)  
        {  
            Console.Write(mensagem);  
            string leitura = LeEntrada().Trim();  
            if (!string.IsNullOrEmpty(leitura))  
            {  
                return leitura;  
            }  
            Console.WriteLine("\nValor inválido! O campo não pode ser  
vazio.");  
        }  
    }  
  
    static int LerInteiro(string mensagem)  
    {  
        while (true)  
        {  
            Console.Write(mensagem);  
            string entrada = LeEntrada().Trim();  
  
            if (int.TryParse(entrada, out int resultado))  
            {  
                return resultado;  
            }  
            Console.WriteLine("\nERRO! Digite um valor válido!");  
        }  
    }  
}
```

```

    }

}

static decimal LerDecimal(string mensagem)
{
    while (true)
    {
        Console.Write(mensagem);
        string entrada = LeEntrada().Trim();
        entrada = separadorDoSistema == "," ? entrada.Replace(".",
",") : entrada.Replace(",", ".");

        if (!decimal.TryParse(entrada, out decimal resultado))
        {
            Console.WriteLine("\nERRO! Digite um número!");
        }
        else if (resultado < 0)
        {
            Console.WriteLine("\nERRO! Digite um valor maior ou
igual a 0!");
        }
        else
        {
            return resultado;
        }
    }
}

static int Menu()
{
    Console.WriteLine("\n=====> MENU <=====");
    Console.WriteLine("0 - Sair");
    Console.WriteLine("1 - Cadastrar preço");
    Console.WriteLine("2 - Consultar preço");
    Console.WriteLine("=====> MENU <=====");
    return LerInteiro("Digite sua opção: ");
}

static void CadastrarPreco(Dictionary<string, decimal> produtos)
{
    string codigo = LeStringObrigatoria("\nDigite o código do
produto: ").ToUpper();

```

```

        if (produtos.ContainsKey(codigo))
        {
            Console.WriteLine($"\\nERRO! Produto com o código
\\\"{codigo}\\\" já existe.");
            return;
        }

        decimal preco = LerDecimal("Digite o preço do produto: ");
        produtos.Add(codigo, preco);
        Console.WriteLine("\\nPRODUTO CADASTRADO COM SUCESSO!");
    }

    static void ConsultarPreco(Dictionary<string, decimal> produtos)
    {
        string codigo = LeStringObrigatoria("\\nDigite o código do
produto: ").ToUpper();
        if (produtos.TryGetValue(codigo, out decimal preco))
        {
            Console.WriteLine($"\\nProduto encontrado (Código: {codigo}
| Preço: {preco:C})");
        }
        else
        {
            Console.WriteLine($"\\nERRO! Produto com o código
\\\"{codigo}\\\" não foi encontrado.");
        }
    }

    static void Main()
    {
        Console.Clear();
        Console.WriteLine("=== CONSULTA DE PREÇOS ===\\n");
        Console.WriteLine("Este programa aceita vírgula ou ponto como
separador decimal.");

        Dictionary<string, decimal> produtos = new Dictionary<string,
decimal>();

        while (true)
        {
            int opcao = Menu();
            switch (opcao)

```

```
        {
            case 0:
                Console.WriteLine("\n--> PROGRAMA ENCERRADO
<--\n");

                return;
            case 1:
                CadastrarPreco(produtos);
                break;
            case 2:
                ConsultarPreco(produtos);
                break;
            default:
                Console.WriteLine("\nOPÇÃO INVÁLIDA\n");
                break;
        }
    }
}
```